LAB WRITEUP 06 - HTTP IN DETAIL (TRYHACKME)

Room	HTTP in Detail
Platform	TryHackMe - `https://tryhackme.com/room/httpindetail`
Path	Pre Security / Cyber Security 101 - How the Web Works
Difficulty	Easy
Time to complete	About 60 minutes
Tasks	7

Objective

Understand how a browser actually talks to a web server: the request/response model, URLs, HTTP methods, status codes, request and response headers, cookies, and making real requests.

Concepts covered

- HTTP vs HTTPS, and what TLS actually protects
- Anatomy of a URL (scheme, user, host, port, path, query string, fragment)
- Methods: GET, POST, PUT, DELETE (and TRACE)
- Status code families and the most common codes
- Request and response headers, cookies and state
- Hands-on requests against the room's web server

Task 1 - What is HTTP(S)?

HTTP (HyperText Transfer Protocol) is the application-layer protocol a browser uses to request content from a web server and receive it in return. It is stateless - each request stands on its own - and it carries no encryption on its own.

HTTPS is HTTP over TLS: the content is encrypted in transit, and the server is validated by a certificate, so a user can tell they are talking to the real site and nobody on the path can read the traffic.

Question	Answer	How
What does HTTP stand for?	HyperText Transfer Protocol	-
What does the S in HTTPS stand for?	Secure	It means the connection is protected by TLS.
Flag on the mock page where the certificate is invalid	THM{INVALID_HTTP_CERT}	Click the padlock in the mock browser's URL bar. A certificate that does not match the site is precisely how users are tricked into accepting a man-in-the-middle.

Task 2 - Requests and Responses

Every exchange is one request and one response. A URL is the instruction that says what to fetch and how:

Part	Meaning	Example
Scheme	Which protocol to use	`http://` or `https://`
User	Optional credentials in the URL	`user:password@`
Host	Domain name or IP of the server	`tryhackme.com`
Port	80 for HTTP, 443 for HTTPS by default	`:8080`
Path	The resource being requested	`/blog`
Query string	Extra parameters, as key=value pairs	`?id=1`
Fragment	A reference to a section within the page	`#section-2`

Question	Answer	How
Which HTTP protocol version is used in the example?	HTTP/1.1	Read the request line at the top of the example request.
Which response header tells the browser how much data to expect?	Content-Length	It states the size of the response body in bytes.

Task 3 - HTTP Methods

Methods express the intended action:

Method	Purpose
GET	Retrieve data from the server
POST	Submit data and typically create a new record
PUT	Submit data to update an existing record
DELETE	Remove a record or resource
TRACE	Loop-back test for debugging (often disabled)

Question	Answer
Method used to create a new user account	POST
Method used to update your email address	PUT
Method used to remove a picture you uploaded	DELETE
Method used to view a news article	GET

SOC relevance: a flood of `POST` requests to a login endpoint is credential guessing; a `GET` request containing `../` or `%2E%2E` in the path is path traversal; unexpected `PUT` or `DELETE` requests on an application that should only serve content are a misconfiguration worth escalating.

Task 4 - HTTP Status Codes

Range	Family	Meaning
100-199	Informational	Request received, continue
200-299	Success	The request was handled
300-399	Redirection	Look somewhere else

Range	Family	Meaning
400-499	Client error	The request itself was wrong
500-599	Server error	The server failed to handle a valid request

Question	Answer	Code meaning
Response code after creating a new user or blog post	201	201 Created
Response code when accessing a page that does not exist	404	404 Not Found
Response code when the web server cannot reach its database and the application crashes	503	503 Service Unavailable
Response code when trying to edit a profile without logging in first	401	401 Not Authorised

Other codes worth knowing by heart: 200 OK, 301/302 redirect, 400 Bad Request, 403 Forbidden (authenticated but not permitted), 405 Method Not Allowed, 500 Internal Server Error.

Task 5 - Headers

Headers are the key/value metadata attached to a request or a response.

Common request headers: `Host`, `User-Agent`, `Content-Length`, `Accept-Encoding`, `Cookie`. Common response headers: `Set-Cookie`, `Cache-Control`, `Content-Type`, `Content-Encoding`.

Question	Answer	Why it matters
Which header tells the web server what browser is being used?	User-Agent	A User-Agent of `curl/8.x` or `sqlmap/1.7` where a browser should be is a strong indicator of automated tooling.
Which header tells the browser what type of data is being returned?	Content-Type	It decides how the browser interprets the body (`text/html`, `application/json`, `image/png`...).
Which header tells the web server which website is being requested?	Host	One server can host many sites, and the `Host` header selects between them - which is why it is also used for virtual-host confusion attacks.

Task 6 - Cookies

Because HTTP is stateless, cookies are the mechanism that gives a session continuity. The server sends `Set-Cookie`, and the browser then includes `Cookie` on every subsequent request.

Question	Answer
Which header is used to save cookies to your computer?	Set-Cookie

The cookie value is normally an opaque token, not a password in clear text, but that token is the session - so stealing a cookie is equivalent to stealing the login. That is why cookie theft, session hijacking (`T1539 Steal Web Session Cookie`) and flags such as `HttpOnly` and `Secure` matter to a SOC.

Task 7 - Making Requests

The room provides a mock web server and a request tool. Each flag proves the request was actually made correctly.

Request made	Flag returned
`GET /room`	THM{YOU'RE_IN_THE_ROOM}
`GET /blog?id=1` (query parameter set with the gear icon)	THM{YOU_FOUND_THE_BLOG}
`DELETE /user/1`	THM{USER_IS_DELETED}
`PUT /user/2` with `username=admin`	THM{USER_HAS_UPDATED}
`POST /login` with `username=thm` and `password=letmein`	THM{HTTP_REQUEST_MASTER}

What this proves: the method, the path and the parameters together fully define a request - and changing any one of them changes what the server does. On the defensive side that is exactly why web-server access logs (method, path, status, user-agent) are so valuable.

Key takeaways

1. HTTP is request/response and stateless; HTTPS adds TLS for confidentiality and server identity.
2. A URL is a precise instruction: scheme, host, port, path, query string and fragment.
3. Methods carry intent - GET reads, POST creates, PUT updates, DELETE removes.
4. Status codes tell the story of the request - and 401 with a burst of attempts is a brute force indicator in any web log.
5. Headers are detection gold: a wrong `User-Agent`, a mismatched `Content-Type` or an unexpected `Host` are all cheap, high-signal signs of tooling and misbehaviour.
6. Cookies are sessions - protecting them (`HttpOnly`, `Secure`, short expiry) is protecting the user's account.